

Aim:

7. Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

Code:

```
import breeze.linalg.{DenseVector, euclideanDistance}
import scala.io.Source

object S091_Shreyash_M2_P7 {
  case class DataPoint(features: DenseVector[Double], label: String)

  def main(args: Array[String]): Unit = {

    println("Shreyash Kadam S091")

    val stream = getClass.getResourceAsStream("/all_stocks_5yr.csv")

    if (stream == null) {
      println("Error: File not found in resources folder!")
      return
    }

    val file = Source.fromInputStream(stream)

    val dataset = file.getLines().drop(1).flatMap { line =>
      val cols = line.split(",")

      if (cols.length >= 7 && cols(6).trim == "AAL") {
        for {
          open <- cols(1).trim.toDoubleOption
          close <- cols(4).trim.toDoubleOption
        } yield {
          val label =
            if (close > open) "Increase"
            else "No Increase"

          DataPoint(
            DenseVector(open, close),
            label
          )
        }
      } else {
        None
      }
    }
  }
}
```

Shreyash Kadam

S091

```
.take(50).toList

file.close()

println("\nTraining data points:")
dataset.foreach { p =>
  println(s"Features: ${p.features}, Label: ${p.label}")
}

val newPointFeatures = DenseVector(40.0, 40.5)

println(s"\nNew data point to classify: $newPointFeatures")

var minDistance = Double.MaxValue
var predictedLabel = ""

for (point <- dataset) {

  val dist =
    euclideanDistance(newPointFeatures, point.features)

  if (dist < minDistance) {
    minDistance = dist
    predictedLabel = point.label
  }
}

println("\nClassification Result:")
println(s"Nearest neighbor distance: $minDistance")
println(s"Predicted label: $predictedLabel")
}
}
```

SHETH LUJ AND SIR MV COLLEGE
Scala for Data Science M2 - 7 and 8

Output:

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...  
Shreyash Kadam S091  
  
Training data points:  
Features: DenseVector(15.07, 14.75), Label: No Increase  
Features: DenseVector(14.89, 14.46), Label: No Increase  
Features: DenseVector(14.45, 14.27), Label: No Increase  
Features: DenseVector(14.3, 14.66), Label: Increase  
Features: DenseVector(14.94, 13.99), Label: No Increase  
Features: DenseVector(13.93, 14.5), Label: Increase  
Features: DenseVector(14.33, 14.26), Label: No Increase  
Features: DenseVector(14.17, 13.33), Label: No Increase  
Features: DenseVector(13.62, 13.37), Label: No Increase  
Features: DenseVector(13.57, 13.57), Label: No Increase  
Features: DenseVector(13.6, 13.02), Label: No Increase  
Features: DenseVector(13.14, 13.26), Label: Increase  
Features: DenseVector(13.28, 13.41), Label: Increase  
Features: DenseVector(13.49, 13.43), Label: No Increase  
Features: DenseVector(13.37, 13.61), Label: Increase  
Features: DenseVector(13.5, 13.9), Label: Increase  
Features: DenseVector(14.01, 14.05), Label: Increase  
Features: DenseVector(14.52, 14.57), Label: Increase  
Features: DenseVector(14.7, 14.82), Label: Increase  
Features: DenseVector(14.99, 14.92), Label: No Increase  
Features: DenseVector(14.85, 15.13), Label: Increase  
Features: DenseVector(15.14, 15.5), Label: Increase  
Features: DenseVector(15.54, 15.91), Label: Increase  
Features: DenseVector(15.98, 16.25), Label: Increase  
Features: DenseVector(16.45, 15.98), Label: No Increase  
Features: DenseVector(15.8, 16.29), Label: Increase  
Features: DenseVector(16.48, 16.78), Label: Increase  
Features: DenseVector(17.13, 17.23), Label: Increase  
Features: DenseVector(17.21, 17.0), Label: No Increase  
Features: DenseVector(17.1, 16.86), Label: No Increase  
Features: DenseVector(16.92, 16.6), Label: No Increase  
Features: DenseVector(16.67, 16.51), Label: No Increase  
Features: DenseVector(16.48, 16.65), Label: Increase
```

Aim:

8. Cluster a dataset into two groups using k-means clustering in Breeze.

Code:

```
import breeze.linalg._
import scala.io.Source

object S091_Shreyash_M2_P8 {
  def main(args: Array[String]): Unit = {

    println("Shreyash Kadam S091")

    val stream = getClass.getResourceAsStream("/all_stocks_5yr.csv")

    if (stream == null) {
      println("Error: File not found in resources folder!")
      return
    }

    val file = Source.fromInputStream(stream)

    val values = file.getLines().drop(1).flatMap { line =>
      val cols = line.split(",")

      if (cols.length >= 7 && cols(6).trim == "AAL") {
        for {
          open <- cols(1).trim.toDoubleOption
          close <- cols(4).trim.toDoubleOption
        } yield (open, close)
      } else {
        None
      }
    }.take(50).toList

    file.close()

    val data = DenseMatrix(
      values.map { case (open, close) =>
        Array(open, close)
      }: _*
    )

    val k = 2
    val numFeatures = 2
```

```
val maxIterations = 100

println(s"\nDataset with ${data.rows} samples and ${data.cols} features.")

var centroids = DenseMatrix.zeros[Double](k, numFeatures)

centroids(0, ::) := data(0, ::)
centroids(1, ::) := data(25, ::)

println(s"\nInitial centroids:\n$centroids")

var assignments = DenseVector.zeros[Int](data.rows)
var previousAssignments = DenseVector.fill[Int](data.rows)(-1)

var iteration = 0
var converged = false

while (iteration < maxIterations && !converged) {

  println(s"\n--- Iteration ${iteration + 1} ---")

  for (i <- 0 until data.rows) {

    val point = data(i, ::).t

    var minDistance = Double.MaxValue
    var closestCentroidIndex = -1

    for (j <- 0 until k) {

      val centroid = centroids(j, ::).t

      val dist = euclideanDistance(point, centroid)

      if (dist < minDistance) {
        minDistance = dist
        closestCentroidIndex = j
      }
    }

    assignments(i) = closestCentroidIndex
  }

  if (assignments == previousAssignments) {
```

```
    converged = true
  } else {
    previousAssignments = assignments.copy
  }

  val newCentroids = DenseMatrix.zeros[Double](k, numFeatures)
  val clusterCounts = DenseVector.zeros[Int](k)

  for (i <- 0 until data.rows) {

    val clusterId = assignments(i)

    newCentroids(clusterId, ::) :=
      newCentroids(clusterId, ::) + data(i, ::)

    clusterCounts(clusterId) += 1
  }

  for (i <- 0 until k) {

    if (clusterCounts(i) > 0) {
      newCentroids(i, ::) :=
        newCentroids(i, ::) / clusterCounts(i).toDouble
    }
  }

  centroids = newCentroids

  println(s"Updated centroids:\n$centroids")

  iteration += 1
}

println("\n--- Final Results ---")

println(s"K-means algorithm converged in $iteration iterations.")

println(s"\nFinal centroids:\n$centroids")

println(s"\nFinal cluster assignments:\n$assignments")
}
}
```

Output:

[illegible]